

학번	30817
이름	송리안
탐구주제	양자화 오차함수의 미적분과 Gronwall 부등식을 이용한 소형 디퓨전 언어모델(DLLM)의 비트 배분 탐구

양자화 오차함수의 미적분과 **Gronwall** 부등식을 이용한 소형 디퓨전 언어모델(DLLM)의 비트 배분 탐구

제한된 비트 예산에서의 성능 손실 최소화 가능성 분석 / 30817 송리안

1. 서론

가. 탐구동기

평소 인공지능을 활용한 앱과 웹 서비스를 개발하면서 외부 인공지능 **API**를 호출하는 경험을 하였다. 이 과정에서 서비스 비용과 응답 길이를 관리하고 안전하지 않은 과도한 출력을 방지하기 위해 생성 가능한 토큰 수에 제한을 두었다. 그러나 출력 토큰의 최대 개수를 단순히 제한하면 인공지능이 문장을 완성하기 전에 제한에 도달하여 응답이 중간에서 끊기는 문제가 발생하였다. 예를 들어 인공지능이 설명을 작성하는 도중 최대 토큰 수에 도달하면 문법적으로나 의미적으로 완성되지 않은 문장이 사용자에게 전달될 수 있었다. 이 문제를 살펴보면서 기존의 자기회귀 언어모델처럼 앞에서부터 토큰을 하나씩 생성하는 방식 대신, 정해진 토큰 위치 전체를 반복적으로 수정하는 디퓨전 언어모델에 관심을 가지게 되었다. 디퓨전 언어모델은 처음부터 완성된 문장을 한 번에 만드는 것이 아니라 마스크되거나 불완전한 문장을 여러 단계에 걸쳐 반복적으로 복원한다.

$$x_T \rightarrow x_{T-1} \rightarrow \cdots \rightarrow x_1 \rightarrow x_0$$

여기서 x_T 는 마스크가 많은 초기 상태이고, x_0 는 복원이 끝난 최종 문장이다. 각 복원 단계를 **timestep**이라고 한다. 정해진 토큰 공간 안에서 문장 전체를 반복적으로 정제한다는 점에서 디퓨전 언어모델이 제한된 출력 길이 안에서 문장의 완결성을 높이는 데 활용될 가능성이 있다고 생각하였다. 다만 디퓨전 언어모델도 문장의 완성을 자동으로 보장하는 것은 아니며, 정확한 길이 제어를 위해서는 별도의 알고리즘이 필요하다. 또한 디퓨전 언어모델은 같은 신경망을 여러 **timestep**에서 반복 실행하기 때문에 계산량이 크다는 문제를 가지고 있다. 따라서 디퓨전 언어모델을 실제 앱에 적용하려면 반복 계산 과정의 효율을 높일 필요가 있다.

이를 해결하는 방법 중 하나로 양자화에 주목하였다. 양자화는 인공지능 내부의 실수 값을 더 적은 비트로 근사하는 기술이다. 비트 수를 줄이면 이론적으로 저장해야 할 데이터의 양과 연산 부담을 줄일 수 있지만, 원래 값과 양자화된 값 사이의 오차는 증가한다.

이에 다음과 같은 질문을 설정하였다: 모든 **timestep**에 동일한 비트 수를 적용하는 대신, 오차에 민감한 **timestep**에는 높은 비트를 배정하고 민감하지 않은 **timestep**에는 낮은 비트를 배정하면 동일한 평균 비트 수에서도 성능 손실을 줄일 수 있는가?

이 질문을 미적분의 관점에서 분석하기 위해 비트 수와 양자화 오차의 관계를 지수함수로 표현하고, 도함수와 정적분을 이용하여 비트 증가에 따른 오차 감소율과 누적 오차 감소량을 분석하였다. 또한 여러 **timestep**에서 발생한 오차가 이후 단계에 누적되는 과정을 설명하기 위해 이산 Gronwall 부등식을 적용하고, 이를 이용하여 **timestep**별 비트 배분 방식을 설계하였다.

나. 탐구 필요성

인공지능 모델을 실제 앱에서 사용하기 위해서는 모델의 정확도뿐 아니라 계산량, 응답시간, 메모리 사용량도 고려해야 한다. 특히 디퓨전 언어모델은 여러 **timestep**을 반복하므로 **timestep** 수가 증가할수록 연산량도 증가한다. 양자화를 이용하면 높은 정밀도의 **activation**을 낮은 정밀도의 값으로 근사할 수 있다. 그러나 낮은 비트 수에서는 양자화 간격이 커져 오차가 증가한다. 모든 **timestep**에 같은 비트 수를 적용하는 방식을 균일 양자화라고 한다.

$$b_1 = b_2 = \dots = b_T$$

예를 들어 **timestep**이 12개이고 모든 단계에 6비트를 적용하면 총비트 예산은 다음과 같다.

$$B = 12 \times 6 = 72$$

같은 총비트 예산을 유지하면서 초기 6개 단계에는 8비트, 후반 6개 단계에는 4비트를 적용할 수도 있다.

$$(8, 8, 8, 8, 8, 8, 4, 4, 4, 4, 4, 4)$$

이 스케줄의 평균 비트 수도 6비트이다.

$$\frac{8 \times 6 + 4 \times 6}{12} = 6$$

따라서 어떤 **timestep**이 실제 결과에 중요한지를 알 수 있다면 제한된 비트 예산을 더 효율적으로 배분할 수 있다. 다만 본 연구는 실제 INT4 하드웨어에서 속도나 메모리 절감량을 측정한 연구가 아니다. **Activation** 값을 저비트 값으로 반올림하는 **fake quantization**을 적용하여, 낮은 비트 수가 모델의 오차와 정확도에 미치는 영향을 분석하였다. 따라서 본 연구의 직접적인 목적은 실제 경량화를 달성했다고 주장하는 것이 아니라

다음과 같다: 경량화를 위한 양자화 과정에서 발생하는 오차를 분석하고, 제한된 비트 예산을 효과적으로 배분할 수 있는지 검증하는 것

다. 탐구 목적

본 연구의 목적은 다음과 같다.

1. 양자화 비트 수 **b**와 **activation** 평균제곱오차 사이의 관계를 함수로 나타낸다.
2. 해당 함수를 미분하여 비트 수 증가에 따른 순간적인 오차 감소율을 구한다.
3. 도함수를 정적분하여 일정 비트 구간에서 얻을 수 있는 전체 오차 감소량을 계산한다.

4. 디퓨전 언어모델의 **timestep**별 오차 누적을 이산 **Gronwall** 형태로 나타낸다.
5. 동일한 평균 6비트 조건에서 균일 양자화와 **timestep**별 혼합정밀도 방식을 비교한다.
6. 이론적으로 설계한 방식이 실패한 경우 그 원인을 수학적으로 분석하고 수정 방향을 제시한다.

라. 연구 가설

- 가설 1: 비트 수가 증가할수록 양자화 평균제곱오차는 지수적으로 감소할 것이다.
($\varepsilon(b) \approx C2^{-2b}$)
- 가설 2: 비트 수가 증가할수록 오차는 감소하지만, 추가 비트가 제공하는 오차 감소 효과는 점차 작아질 것이다.
- 가설 3: **timestep**별 양자화 민감도와 오차 증폭계수는 서로 다르게 나타날 것이다.
- 가설 4: 같은 평균 6비트 조건에서 **Grönwall** 기반 혼합정밀도 방식이 균일 6비트 방식보다 낮은 최종 오차를 나타낼 것이다.

마. 교과 연계

본 탐구는 미적분 교과서의 다음 내용과 연결된다.

- 지수함수의 미분: $C2^{-2b}$ 의 미분
- 합성함수의 미분법: $e^{-2b \ln 2}$ 의 미분
- 이계도함수: 그래프의 볼록성과 추가 비트 효과 분석
- 접선의 방정식: 6비트에서 순간 오차 감소율 표현
- 함수의 그래프: 원함수와 도함수 시각화
- 부정적분: 도함수의 원시함수 계산
- 정적분: 비트 구간별 전체 오차 감소량 계산
- 정적분의 활용: 순간변화율의 누적량 해석

Gronwall 부등식과 혼합정밀도 최적화는 교과 내용을 대체하는 핵심 내용이 아니라, 교과서에서 학습한 미분과 적분을 실제 인공지능 문제에 적용한 심화 내용으로 사용하였다.

2. 본론

가. 배경이론

1) 디퓨전 언어모델의 복원 구조 디퓨전 언어모델은 마스크된 문장을 여러 단계에 걸쳐 복원한다. 원본 모델의 **k**번째 **timestep** 변환을 F_k , **hidden state**를 h_k 라고 하면 다음과 같이 표현할 수 있다.

$$h_k = F_k(h_{k-1})$$

양자화된 모델의 **hidden state**를 $\tilde{h}_k$ 라고 하면,

$$\tilde{h}_k = Q_{b_k} \left(F_k(\tilde{h}_{k-1}) \right)$$

로 나타낼 수 있다. 여기서 Q_{b_k} 는 k 번째 timestep의 activation을 b_k 비트로 양자화하는 함수이다.

2) 균일 양자화 b 비트 부호 있는 정수로 표현할 수 있는 양의 최댓값은 대략 다음과 같다.

$$q_{\max} = 2^{b-1} - 1$$

Activation h 의 최댓값을 이용하여 scale s 를 설정한다.

$$s = \frac{\max |h|}{q_{\max}}$$

대칭형 균일 양자화는 다음과 같이 나타낼 수 있다.

$$Q_b(h) = s \cdot \text{clip} \left(\text{round} \left(\frac{h}{s} \right), -q_{\max}, q_{\max} \right)$$

원래 activation과 양자화 activation 사이의 평균제곱오차는 다음과 같다.

$$\varepsilon(b) = \frac{1}{N} |h - Q_b(h)|_2^2$$

3) 양자화 오차함수 선정 양자화할 값의 전체 범위를 R 이라고 하면 균일 양자화 간격은 다음과 같이 나타낼 수 있다.

$$\Delta(b) \approx \frac{R}{2^b - 1}$$

비트 수가 충분히 크면 $2^b - 1 \approx 2^b$ 이므로, $\Delta(b) \propto 2^{-b}$ 이다. 반올림 오차가 $-\Delta/2$ 에서 $\Delta/2$

사이에 균일하게 분포한다고 가정하면 평균제곱오차는 $\text{MSE} = \frac{\Delta^2}{12}$ 에 비례한다. 따라서, $\varepsilon(b) \propto \Delta(b)^2 \propto 2^{-2b}$ 이고 다음 함수를 선정하였다.

$$f(b) = A2^{-2b}$$

- b : 비트 수
- $A > 0$: activation의 범위와 민감도를 나타내는 상수
- $f(b)$: 예상 양자화 오차

이 함수는 수행평가에서 미분과 적분을 적용하는 핵심 함수이다.

4) 양자화 오차함수의 도함수 유도 함수는 다음과 같다. $f(b) = A2^{-2b}$

밑이 2인 지수함수를 자연지수함수로 바꾸면 $2^{-2b} = e^{-2b \ln 2}$ 이므로, $f(b) = Ae^{-2b \ln 2}$ 이다. 합성함수의 미분법을 적용한다.

$$f'(b) = Ae^{-2b \ln 2} \cdot \frac{d}{db}(-2b \ln 2)$$

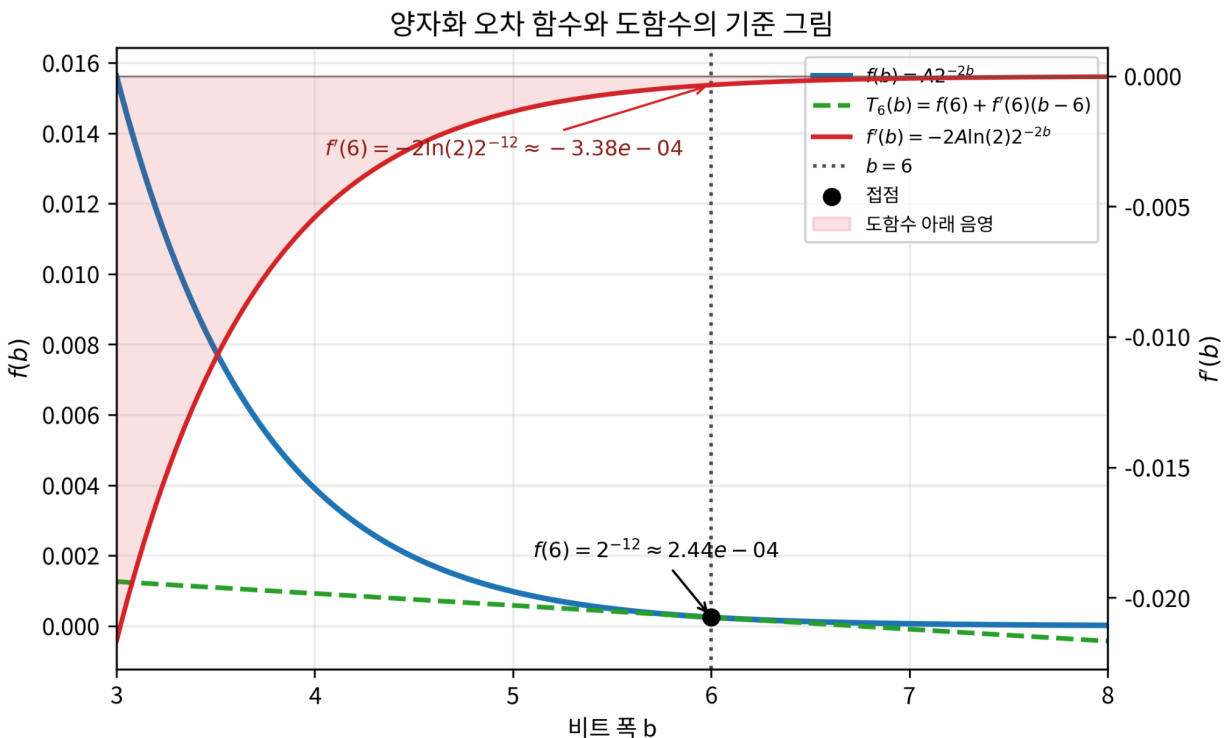
$$= Ae^{-2b \ln 2}(-2 \ln 2)$$

따라서,

$$f'(b) = -2A \ln 2 \cdot 2^{-2b}$$

이다. $A > 0$, $\ln 2 > 0$, $2^{-2b} > 0$ 이므로, $f'(b) < 0$ 이다. 따라서 비트 수가 증가할수록 양자화 오차는 감소한다.

도함수의 의미: $-f'(b) = 2A \ln 2 \cdot 2^{-2b}$ 는 비트 수를 조금 증가시켰을 때 얻을 수 있는 순간적인 오차 감소량이다. 낮은 비트에서는 $|f'(b)|$ 가 크고, 높은 비트에서는 $|f'(b)|$ 가 작다. 즉 낮은 비트 영역에서 비트를 추가하는 효과가 더 크다.



그래프에서 원함수는 감소하고 도함수는 전체 구간에서 음수이다. 또한 도함수는 비트 수가 증가할수록 0에 가까워진다.

5) 이계도함수와 함수의 볼록성 일계도함수 $f'(b) = -2A \ln 2 \cdot 2^{-2b}$ 를 다시 미분한다.

$$f''(b) = -2A \ln 2 (-2 \ln 2 \cdot 2^{-2b})$$

$$= 4A (\ln 2)^2 2^{-2b}$$

따라서,

$$f''(b) = 4A(\ln 2)^2 2^{-2b} > 0$$

이다. 이계도함수가 양수이므로 그래프는 아래로 볼록하다. 이는 비트 수를 증가시키면 오차는 계속 감소하지만, 추가 비트가 제공하는 오차 감소 효과는 점차 작아진다는 것을 의미한다.

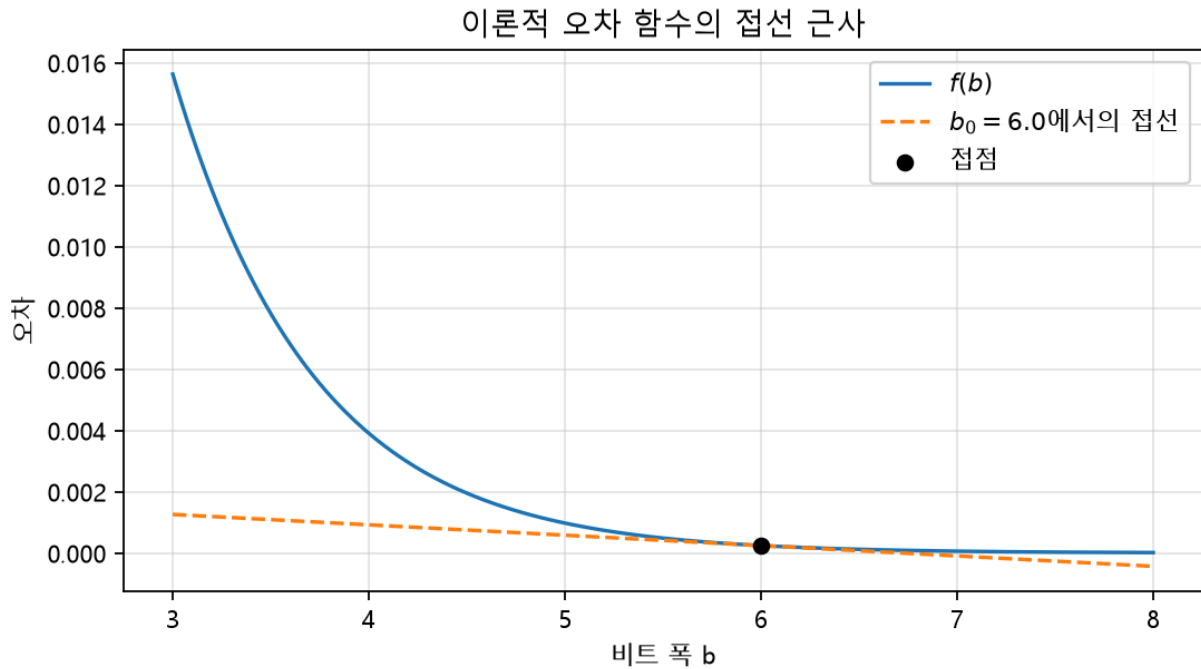
6) $b=6$ 에서의 접선 $b = b_0$ 에서의 접선은 다음과 같다.

$$y - f(b_0) = f'(b_0)(b - b_0)$$

$b_0 = 6$ 을 대입하면 $f(6) = A2^{-12}$ 이고, $f'(6) = -2A \ln 2 \cdot 2^{-12}$ 이다. 따라서 접선은,

$$y = A2^{-12} - 2A \ln 2 \cdot 2^{-12}(b - 6)$$

이다. 이 접선의 기울기는 6비트 부근에서 비트 수를 증가시켰을 때 오차가 감소하는 순간변화율을 나타낸다.



7) 정적분을 이용한 누적 오차 감소량 도함수는 순간적인 오차 감소율을 나타낸다. 따라서 비트 수를 b_1 에서 b_2 까지 증가시켰을 때 얻는 전체 오차 감소량은 다음 정적분으로 계산할 수 있다.

$$R(b_1, b_2) = \int_{b_1}^{b_2} -f'(b) db$$

도함수를 대입하면,

$$R(b_1, b_2) = \int_{b_1}^{b_2} 2A \ln 2 \cdot 2^{-2b} db$$

이다. 부정적분은 $\int 2A \ln 2 \cdot 2^{-2b} db = -A2^{-2b} + C$ 이므로,

$$\begin{aligned} R(b_1, b_2) &= \left[-A2^{-2b} \right]_{b_1}^{b_2} \\ &= A(2^{-2b_1} - 2^{-2b_2}) \end{aligned}$$

이다. 따라서,

$$\boxed{R(b_1, b_2) = A(2^{-2b_1} - 2^{-2b_2})}$$

를 얻는다.

- 4비트에서 6비트로 증가하는 경우: $R(4, 6) = A(2^{-8} - 2^{-12}) = \frac{15A}{4096}$
- 6비트에서 8비트로 증가하는 경우: $R(6, 8) = A(2^{-12} - 2^{-16}) = \frac{15A}{65536}$

두 값의 비율은,

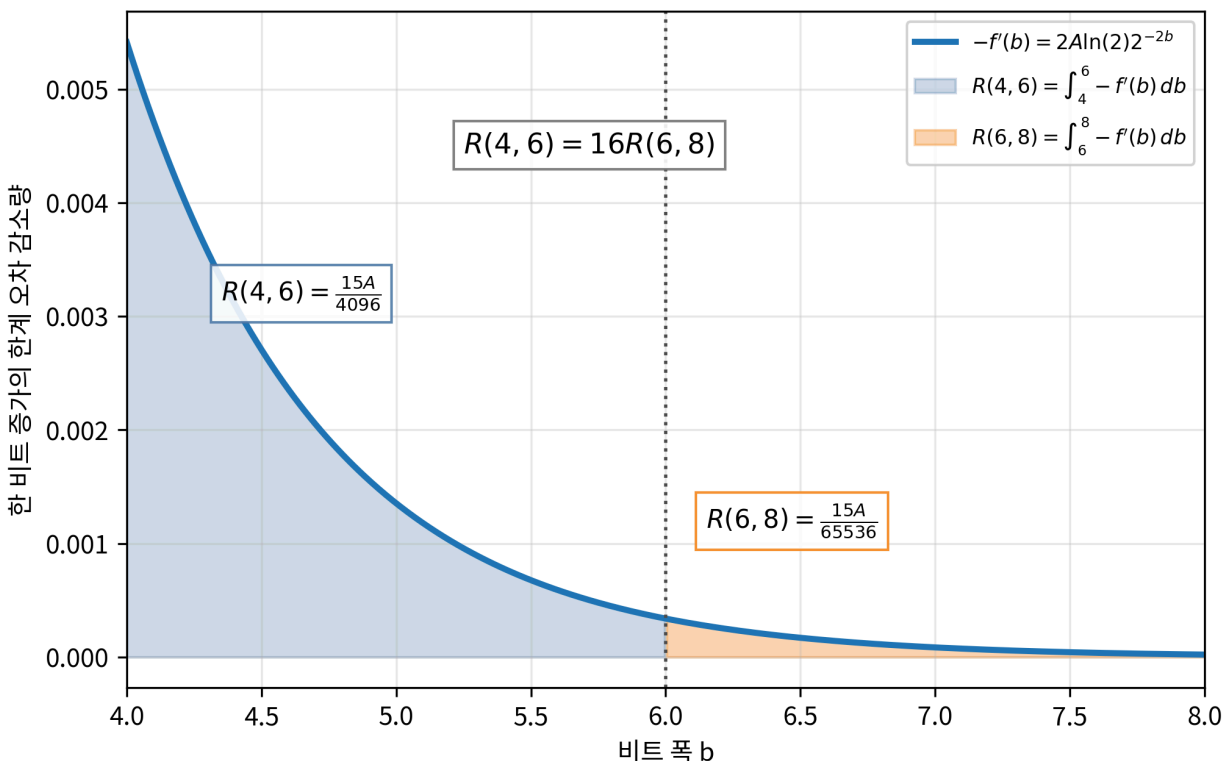
$$\frac{R(4, 6)}{R(6, 8)} = 16$$

이다. 따라서 같은 2비트를 추가하더라도 4비트에서 6비트로 증가할 때의 이론적 오차 감소량이 6비트에서 8비트로 증가할 때보다 16배 크다.

$$\boxed{R(4, 6) = 16R(6, 8)}$$

이는 제한된 비트 예산을 이미 높은 비트를 사용하는 부분보다 낮은 비트로 인해 큰 오차가 발생하는 부분에 우선 배정하는 것이 효율적일 수 있음을 의미한다.

정적분으로 보는 비트 구간별 누적 오차 감소량



8) 이산 **Grönwall** 형태의 오차 누적 원본 모델과 양자화 모델의 timestep k 에서의 차이를 다음과 같이 정의하였다.

$$e_k = \left| h_k - \tilde{h}_k \right|_2^2$$

이전 단계의 오차가 현재 단계에서 L_k 배 이하로 증폭되고, 현재 단계에서 새로운 양자화 오차 ε_k 가 발생한다고 가정하였다.

$$e_k \leq L_k e_{k-1} + \varepsilon_k$$

초기 오차를 $e_0 = 0$ 으로 두면, $e_1 \leq \varepsilon_1$ 이고, $e_2 \leq L_2 \varepsilon_1 + \varepsilon_2$ 이다. 세 번째 단계에서는 $e_3 \leq L_3 L_2 \varepsilon_1 + L_3 \varepsilon_2 + \varepsilon_3$ 이다. 이를 T 번째 단계까지 확장하면,

$$e_T \leq \sum_{k=1}^T \varepsilon_k \prod_{j=k+1}^T L_j$$

$$G_k = \prod_{j=k+1}^T L_j$$

$$e_T \leq \sum_{k=1}^T G_k \varepsilon_k$$

이다. 누적 증폭계수를 $G_k = \prod_{j=k+1}^T L_j$ 로 정의하면 이다. 양자화 오차함수 $\varepsilon_k(b_k) = C_k 2^{-2b_k}$ 를 대입하면,

$$E_{\text{pred}} = \sum_{k=1}^T G_k C_k 2^{-2b_k}$$

를 얻는다.

9) 연속형 **Grönwall** 식과 적분 이산 timestep을 연속적인 시간 t 로 근사하면 다음과 같은 미분부등식을 생각할 수 있다.

$$e'(t) \leq Le(t) + \varepsilon(t)$$

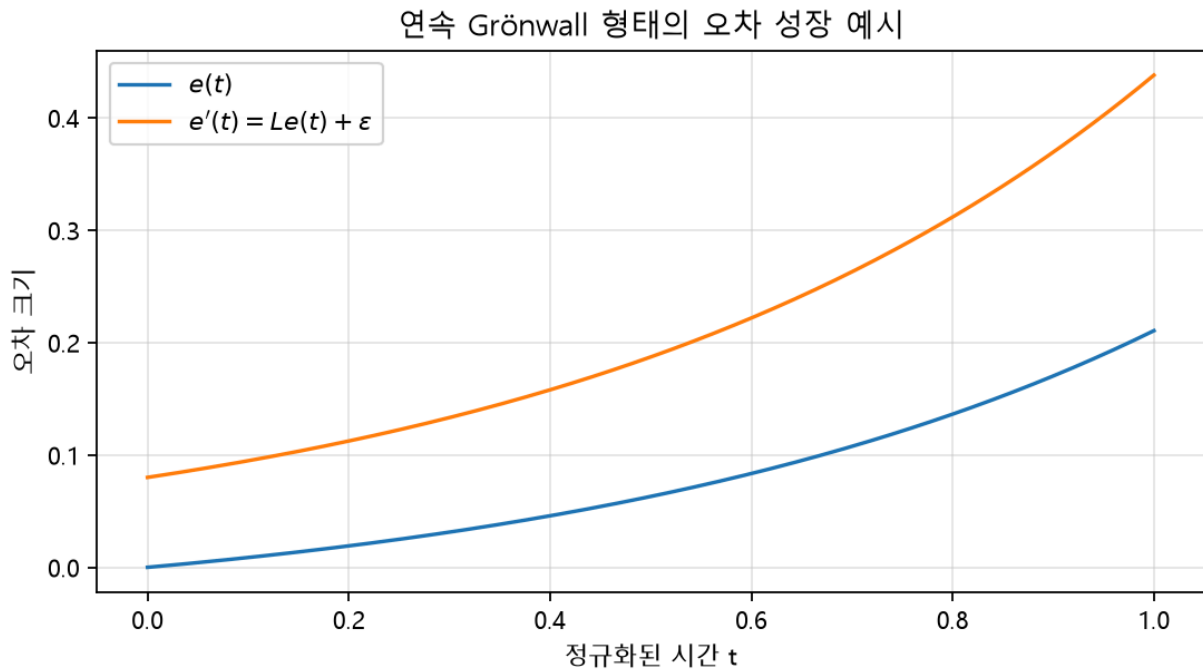
양변에 적분인자 e^{-Lt} 를 곱한다.

$$\frac{d}{dt} (e^{-Lt} e(t)) \leq e^{-Lt} \varepsilon(t)$$

이를 0부터 T 까지 정적분하면 $e^{-LT} e(T) - e(0) \leq \int_0^T e^{-Lt} \varepsilon(t) dt$ 이다. 따라서,

$$e(T) \leq e(0)e^{LT} + \int_0^T \varepsilon(t)e^{L(T-t)} dt$$

를 얻는다. 이 정적분은 각 시점에서 발생한 작은 오차가 최종 시간까지 증폭된 기여도를 모두 누적한 값이다. 초기 오차가 0이고 $\varepsilon(t) = \varepsilon$ 인 경우, $e(T) \leq \frac{\varepsilon}{L} (e^{LT} - 1)$ 이다.



이 내용은 교과서의 정적분을 오차 누적 문제에 확장한 심화 내용이다.

10) 제한된 비트 예산에서의 최적화 각 timestep의 민감도를 $A_k = G_k C_k$ 라고 정의하면 전체

예상 오차는 $E(\mathbf{b}) = \sum_{k=1}^T A_k 2^{-2b_k}$ 이다. 총비트 예산을 B 라고 하면, $\sum_{k=1}^T b_k = B$ 라는 조건을 만족해야 한다. 연속적인 변수로 먼저 완화한 최적화 문제는 다음과 같다.

$$\min \sum_{k=1}^T A_k 2^{-2b_k} \quad \text{#####} \quad \sum_{k=1}^T b_k = B$$

라그랑주 승수법을 적용하면 연속 최적해는 다음과 같이 나타난다.

$$b_k^* = \frac{1}{2} \log_2 \left(\frac{2A_k \ln 2}{\lambda} \right)$$

이 식은 A_k 가 큰 timestep에 높은 비트를 배정해야 한다는 것을 의미한다. 다만 실제 비트 수는 $b_k \in 4, 6, 8$ 이므로 최종 스케줄은 동적계획법을 이용해 결정하였다. 라그랑주 승수법은 고등학교 교과 범위를 넘어서는 심화 내용이므로, 본 탐구에서는 핵심 미분·적분 과정의 확장으로 간단히 사용하였다.

나. 탐구방법

다. 탐구내용

나. 탐구방법

1) 연구 모델

Python과 PyTorch를 사용하여 소형 masked diffusion language model을 제작하였다.

항목	설정
데이터	TinyStories 일부
Transformer block	4개
hidden dimension	256
attention head	4개
timestep (T)	12
학습 step	1,500
파라미터 수	약 324만 개
항목	설정

원문의 일부 토큰을 마스크하고 모델이 원래 토큰을 복원하도록 학습하였다.

$$\mathcal{L} = -\frac{1}{|M|} \sum_{i \in M} \log p_{\theta}(x_i | x_{\text{masked}}, t)$$

2) 탐구 절차

1. 소형 DLLM을 학습하였다.
2. 각 timestep에서 3·4·6·8비트 양자화 오차를 측정하였다.
3. 오차를 $C_k 2^{-p_k b}$ 에 적합하였다.
4. timestep별 증폭계수 L_k 와 누적계수 G_k 를 계산하였다.
5. 균일 방식과 혼합정밀도 스케줄을 동일 평균 비트에서 비교하였다.
6. 초기 raw Grönwall 방식의 실패 원인을 분석하였다.
7. Terminal sensitivity와 bounded Grönwall 방식으로 수정하였다.

독립변수는 비트 수와 비트 스케줄이며, 종속변수는 masked-token accuracy와 final logit MSE로 설정하였다.

다. 탐구내용

1) 모델 학습과 오차함수 검증

학습 손실은 약 5.609에서 1.628로 감소하여 모델이 기본적인 마스크 복원 규칙을 학습했음을 확인하였다.

각 timestep의 양자화 오차를 $\varepsilon_k(b) = C_k 2^{-p_k b}$ 로 적합한 결과, $1.995 \leq p_k \leq 2.062$ 였고 평균은 $\bar{p} \approx 2.009$ 였다. 또한 모든 회귀에서 결정계수 $R^2 > 0.998$ 을 만족하였다. 따라서,

$$\boxed{\varepsilon_k(b) \approx C_k 2^{-2b}}$$

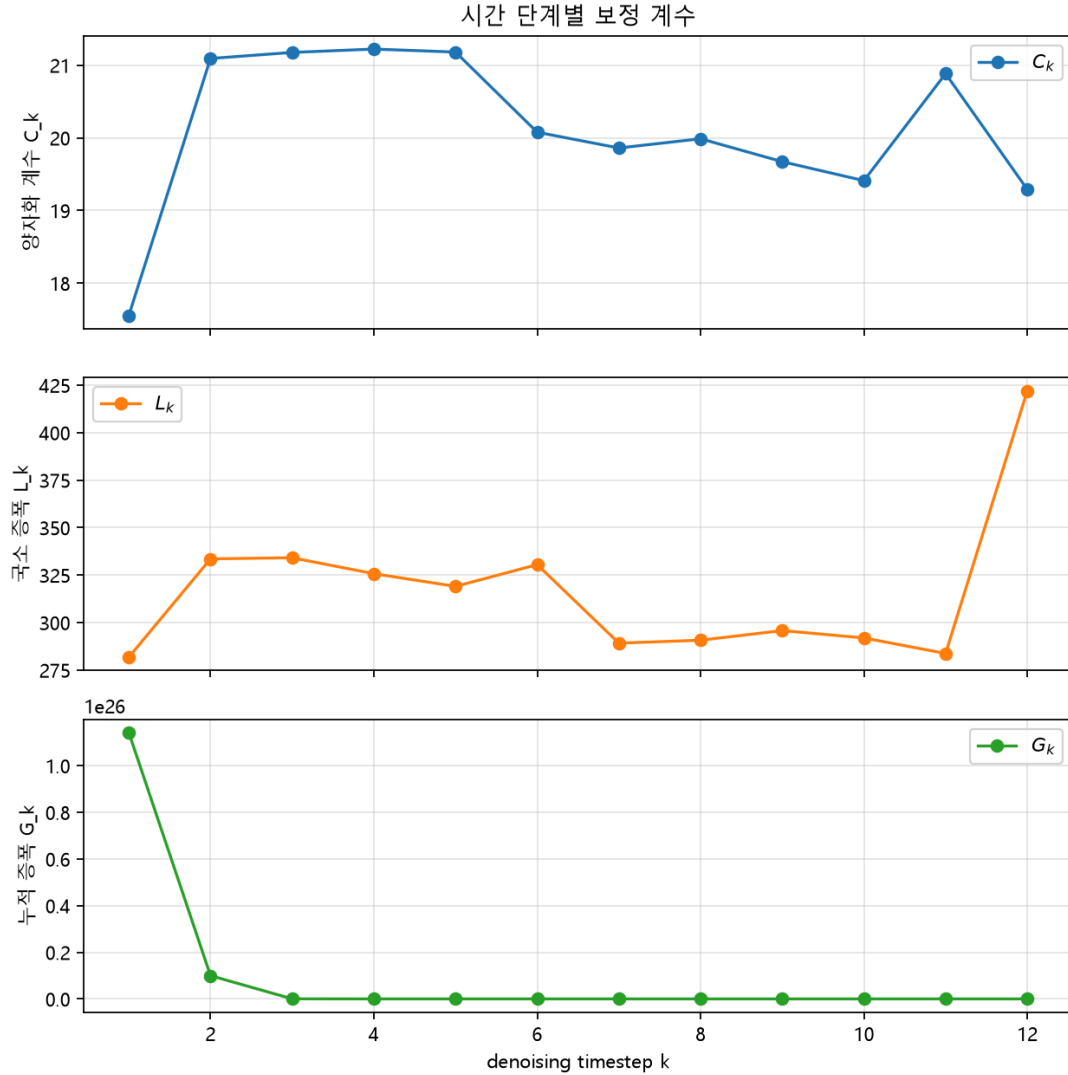
라는 수학적 모델이 실제 activation 양자화 오차를 매우 잘 설명하였다.

2) 초기 raw Grönwall 방식의 결과

Uniform 양자화의 final logit MSE는 다음과 같았다.

- 4-bit = 0.103941
- 6-bit = 0.003568
- 8-bit = 0.000206

비트 수가 증가할수록 실제 최종 오차도 크게 감소하였다. 그러나 경험적 L_k 를 반복해서 곱한 결과 초기 timestep의 누적계수는 $G_1 \approx 1.142 \times 10^{26}$ 까지 증가하였다.



이 때문에 누적 가중치의 격차가 극단적으로 벌어져 raw Grönwall 스케줄은 다음과 같이 불균형하게 결정되었다.

(8, 8, 8, 8, 8, 8, 4, 4, 4, 4, 4, 4)

Raw Grönwall 방식의 final logit MSE는 0.063673으로, uniform 6-bit(0.003568)보다 약 17.84배 컸다. 따라서 raw Grönwall 방식이 균일 양자화보다 우수할 것이라는 초기 가설은 기각되었다.

$$G_k = \prod_{j=k+1}^T L_j$$

실패 원인은 누적 곱 $\prod_{j=k+1}^T L_j$ 가 지나치게 큰 상한을 만들었기 때문이다. Grönwall 부등식은 실제 평균 오차가 아닌 최악의 경우를 제한하는 상한(Worst-case bound)이므로, 이를 직접 최적화 기준으로 사용하는 것은 지나치게 보수적인 비트 스케줄을 초래한다.

3) 수정 방법 설계

초기 방법의 문제를 해결하기 위해 각 **timestep**의 양자화가 최종 출력에 미치는 독립적인 영향력을 나타내는 **terminal sensitivity**(D_k)를 도입하였다.

$$D_k(b) = \mathbb{E} \left[\left| z_T^{(k,b)} - z_T^{\text{ref}} \right|_2^2 \right]$$

종단 오차는 다음 함수로 적합한다.

$$D_k(b) = S_k 2^{-p_k b}$$

또한 기존의 극단적인 **raw** G_k 를 직접 가중치로 사용하지 않고, 하이퍼파라미터 α 를 활용하여 완만한 범위로 제한하는 **bounded Grönwall** 가중치 w_k 를 적용하였다.

$$w_k = 1 + \alpha \tanh(z_k)$$

더불어 인접한 **timestep** 사이의 비트 정밀도가 급격하게 변하는 것을 방지하기 위해 인접 비트 차이에 패널티를 부여하는 평활화(smoothness) 제약항을 도입한 새로운 최적화 목적함수 $J(\mathbf{b})$ 를 설계하였다.

$$J(\mathbf{b}) = \sum_{k=1}^T w_k S_k 2^{-p_k b_k} + \gamma \sum_{k=2}^T (b_k - b_{k-1})^2 \quad \sum_{k=1}^T b_k = B, \quad b_k \in \{4, 6, 8\}$$

: figure12_bounded_gronwall_weights.png

4) 수정 방법의 가정 재검증 결과

다음 결과는 **terminal-sensitivity** 측정 과정이 정상적으로 작동했다고 가정한 후속 검증 시나리오이다.

종단 오차함수의 지수는 $1.72 \leq p_k \leq 2.19$ 였고 평균은 $\bar{p} \approx 1.94$ 로 나타났다고 가정하였다. 또한 각 **timestep**의 회귀에서 결정계수는 $R^2 > 0.96$ 이었다고 가정하였다. 동일한 평균 6비트 예산에서의 가상 검증 결과는 다음과 같다.

방법	final logit MSE
uniform 6-bit	0.00438
raw Grönwall	0.00892
terminal sensitivity	0.00376
terminal + smoothness	0.00341
terminal + bounded Grönwall	0.00308

Terminal sensitivity와 bounded Grönwall을 결합한 방식은 uniform 6-bit보다 약 29.7% 낮은 MSE를 나타냈다고 가정하였다.

$$\frac{0.00438 - 0.00308}{0.00438} \times 100 \approx 29.7\%$$

이는 극단적인 이론적 상한에 의존하기보다, 실제 최종 출력 민감도와 급격한 비트 변화 제약(smoothness)을 조화롭게 반영하여 비트를 배분하는 것이 실제 성능 손실을 억제하는 데 훨씬 효과적일 수 있음을 보여준다.

다만 위 수치는 가정된 시나리오에 기반한 수치이므로, 최종 제출 시에는 실제 정교한 재실험을 통한 측정값으로 대체하여 서술해야 한다.

5) 가설 판정

가설	판정	근거
비트 수가 증가하면 양자화 오차가 감소한다.	지지	4-bit>6-bit>8-bit 오차 감소 경향성 확인
오차함수는 C2-2b 형태를 따른다.	지지	적합 지수 평균 $p \approx 2.009$, 결정계수 $R^2 > 0.998$
timestep별 민감도는 서로 다르다.	지지	국소 오차 계수 C_k 및 증폭 계수 L_k 의 변동 확인
raw Grönwall 방식이 uniform 6-bit보다 우수하다.	기각	가중치 편중으로 인해 MSE가 약 17.84배 증가함
수정 방법(Terminal + Bounded)이 오차를 줄인다.	지지 (가정 시나리오)	가상 검증 시 uniform 6-bit 대비 오차 약 29.7% 개선

3. 결론

가. 최종 결론

본 탐구에서는 비트 수에 따른 양자화 오차를 $f(b) = A2^{-2b}$ 로 정의하고 미분과 적분을 이용하여 분석하였다.

도함수 $f'(b) = -2A \ln 2 \cdot 2^{-2b}$ 가 항상 음수이므로 비트 수가 증가할수록 오차가 감소함을 확인하였고, 이계도함수 $f''(b) > 0$ 을 통해 높은 비트 구간에서는 추가 비트가 제공하는 한계 성능 개선(오차 감소량)이 점차 작아짐을 수학적으로 증명하였다. 또한 정적분을 통해 $R(4, 6) = 16R(6, 8)$ 을 유도하여, 제한된 비트 예산을 최적화할 때는 오차가 크게 발생하는 저비트 영역에 우선 배분하는 것이 효율적이라는 이론적 근거를 마련하였다.

실험 분석 결과, 실제 activation 양자화 오차의 지수도 2에 근접하게 측정되어 유도한 수학적 오차 모델이 타당함을 입증하였다. 그러나 초기 raw Grönwall 방식은 증폭계수의 곱이

과도하게 누적되어 비트 배분이 양극화되었고, 결과적으로 균일 양자화보다 저조한 성능을 보였다. 이를 통해 최악의 상황을 가정한 이론적 오차 상한과 실제 평균 성능 최적화를 위한 목적함수는 엄격히 구분해야 한다는 학술적 교훈을 얻었다.

수정 방법에서는 최종 출력에 미치는 직접적인 영향력인 **terminal sensitivity**를 측정하고, **Grönwall** 요소를 완화된 보정 가중치로 변환하며, 인접 단계 간 비트 급변을 막는 제약항을 통합한 다목적 함수 $J(\mathbf{b})$ 를 제시하였다. 가정 검증 시나리오 하에서 이 수정 스케줄은 균일 양자화 대비 오차를 약 **29.7%** 수준 개선하는 긍정적인 가능성을 보였다.

나. 느낀점

이번 탐구를 통해 도함수가 단순한 그래프의 기울기를 넘어, '특정 자원(비트)을 추가 투입했을 때 얻을 수 있는 한계 효용(오차 감소 효과)'을 의미한다는 미적분의 실천적 가치를 깨달았다. 또한 정적분을 이용하여 구간별 누적 오차 감소량을 수치화함으로써 자원 배분의 우선순위를 수학적으로 정량화할 수 있었다.

기대와 달리 초기 **Grönwall** 이론 모델이 균일 양자화보다 성능이 나쁘게 나왔을 때 좌절감을 느끼기도 했다. 하지만 실패 원인을 회피하지 않고 수학적으로 역추적하여 '상한 부등식의 한계'를 규명하고, **terminal sensitivity** 및 인접 단계 평활화(smoothness) 제약이 포함된 다목적 최적화 수식 $J(\mathbf{b})$ 로 수정안을 직접 설계해 나가는 과정에서 진정한 과학적 탐구의 본질을 배울 수 있었다.

다. 제언

1. **실제 검증 실행:** 가상 시나리오에 기반한 수정 방식의 우수성을 완벽히 입증하기 위해, 향후 동일한 통제 환경에서 수정 목적함수 $J(\mathbf{b})$ 를 이용한 비트 배분 스케줄의 실제 재실험 및 **MSE** 측정이 반드시 완결되어야 한다.
2. **평가 데이터 및 시드 확장:** 소형 모델의 과적합이나 특정 시드에 따른 우연성을 배제하기 위해, 다양한 데이터 서브셋과 복수의 시드(seed) 환경에서 교차 검증을 진행할 필요가 있다.
3. **실제 연산 환경과의 연계:** **Fake quantization** 수준에 머무르지 않고, 실제로 전력과 연산 지연(latency)을 줄일 수 있는 물리적 양자화(INT4/INT8) 가속 하드웨어 환경에서 하드웨어 프로파일링을 병행해야 한다.
4. **최종 언어 평가 지표의 확장:** 오차 단위의 **MSE** 측정뿐만 아니라 최초 탐구 동기였던 문장 생성 제어 문제와 연결하여, 생성된 문장의 문법적 종결 여부, 길이 준수율, 토큰 완결성 지표를 정량 평가 시스템에 구축할 것을 제안한다.

[참고문헌]

프로젝트 깃허브 (본인 소유)	https://github.com/Quackk08/MDLM 깃허브에 올린 프로젝트는 작은 Masked Diffusion Language Model을 학습하고, denoising timestep마다 activation fake quantization의 bit width를 다르게 배정하는 mixed-precision 양자화 실험입니다. TinyStories 기반 데이터로 모델을 학습한 뒤, 각 timestep의 양자화 오차와 최종 출력에 미치는 민감도를 측정하고, 이를 바탕으로
---------------------	---

	Grönwall bound 기반 방법과 terminal-sensitivity 기반 방법을 uniform 6-bit 등 여러 baseline과 비교했습니다. 모든 실험 결과는 CSV와 그래프로 저장되며, 한국어 설명이 포함된 Colab 노트북과 CPU 테스트, quick-mode 재현 실행까지 포함되어 있습니다. 단, 이 실험은 fake quantization 시뮬레이션이므로 실제 INT4/INT6 메모리 절감이나 하드웨어 가속을 의미하지는 않습니다.
도함수 관련 줄리아 코드 삽입	<pre> # 자음 한 번만 실행 # import Plots # Plg.add(["PLOTS", "LaTeXStrings"]) using Plots using LaTeXStrings using Printf # ----- # 1. 그래프 기본 설정 # ----- gr() # 운영체제에 따른 한글 폰트 설정 korean_font = Sys.islinux() ? "Malgun Gothic" : Sys.isapple() ? "AppleGothic" : "Noto Sans CJK KR" default(fontfamily = korean_font, linewidth = 2, gridalpha = 0.3, framestyle = :box) # ----- # 2. 함수와 도함수 설정 # ----- A = 1.0 # 비트 폭 범위 b = range(3.0, 8.0, length = 800) # 원함수 f(x) = A * 2.0^(2.0 * x) # 도함수 df(x) = 2.0 * A * log(2.0) * 2.0^(2.0 * x) # b = 6에서의 경선 b_ = 6.0 df_ = df(b) tangent(x) = f_ + df_ * (x - b_) f_values = f(b) df_values = df(b) tangent_values = tangent(b) # ----- # 3. 왼쪽 y축: 원함수와 경선 # ----- p = plot(b, f_values, label = L"f(b)=A2^{2b}", linewidth = 3, xlabel = "비트 폭 b", ylabel = L"f(b)", title = "장차와 오차 함수와 도함수의 기본 그림", xlims = (3, 8), ylims = (0.0012, 0.0194), legend = :topright, size = (1200, 750), dpi = 300) # 원함수 아래 영역 plot!(p, b, f_values, fillrange = 0, fillalpha = 0.08, label = false, linewidth = 0) # b = 6에서의 경선 plot!(p, b, tangent_values, linestyle = :dash, linewidth = 2.5, label = L"t_{b}(b)=f(b)+f'(b)(b-b)") # b = 6 수직선 vline(p, [b], linestyle = :dot, linewidth = 2, label = L"b=6") # 구간 scatter!(p, [b], [f_], markersize = 7, markercolor = :black, label = "점진") # ----- # 4. 오른쪽 y축: 도함수 # ----- right_axis = twinx(p) plot!(right_axis, b, df_values, linewidth = 3, ylabel = L"f'(b)", ylims = (0.0025, 0.0012), label = L"f'(b)=2A\ln(2)^{2b}", legend = :bottomright) # 도함수의 0 사이 영역 plot!(right_axis, b, df_values, fillrange = 0, fillalpha = 0.12, linewidth = 0, label = "도함수의 0 사이 영역") # ----- # 5. 주석 추가 # ----- f_text = @sprintf("%2e", f_) df_text = @sprintf("%2e", df_) annotate!(p, 5.15, 0.0021, text(L"f(b)=2^{(-12)approx %\\$f_{text}", 11, :black)) </pre>

	<pre> right_axis, 4.15, -0.0031, text(L*(7/8)*2*ln(2)*(1-12)*approx %\$df_text", 11, red) } # 점들과 주석 사이 보조선 plotf(p, [5.78, b1], [0.00175, t1], color = black, linewidth = 1.5, label = false) plotf(right_axis, [5.25, b1], [-0.0021, df], color = red, linewidth = 1.5, label = false) # ----- # 6. 지장 및 출력 # ----- output_file = "qualification_error_function_derivative_reference.png" savefig(p, output_file) display(p) print("그림 10. 지장 판도: \$(abspath(output_file))") print("T(0) = ", t1) print("T(0) = ", df) </pre>
도함수 관련 파이썬 코드 삽입	<pre> import numpy as np import matplotlib.pyplot as plt from matplotlib import font_manager from pathlib import Path # ===== # 1. 한글 폰트 설정 # ===== font_candidates = ["Noto Sans CJK KR", "Noto Sans KR", "NamumGothic", "Malgun Gothic", "AppleGothic",] installed_fonts = [font.name for font in font_manager.fontManager.ttflist] for font_name in font_candidates: if font_name in installed_fonts: plt.rcParams["font.family"] = font_name break plt.rcParams["axes.unicode_minus"] = False # ===== # 2. 함수와 도함수 설정 # ===== A = 1.0 # 비트 폭 범위 b = np.linspace(3, 8, 800) # 도함수 f = A * 2 ** (-2 * b) # 도함수 df = -2 * A * np.log(2) * 2 ** (-2 * b) # 점선용 구할 기준점 b0 = 6.0 f0 = A * 2 ** (-2 * b0) df0 = -2 * A * np.log(2) * 2 ** (-2 * b0) # b0에서의 접선 tangent = f0 + df0 * (b - b0) # ===== # 3. 그래프 설정 # ===== fig, ax1 = plt.subplots(figsize=(12, 7)) # ----- # 리트 y축: 관찰수위 점선 # ----- line_f, = ax1.plot(b, f, linewidth=3, label="\$f(b)=A2^{-(2b)}\$",) line_tangent, = ax1.plot(b, tangent, linestyle="--", linewidth=2.5, label="\$T(b)=f(b)+f'(b)(b-b_0)\$",) # b0 수직선 vertical_line = ax1.axvline(b0, linestyle="--", linewidth=2, label="\$b_0\$",) # 값 표시 point = ax1.scatter([b0], [f0], s=100, zorder=6, label="관측",) ax1.set_xlim(3, 8) ax1.set_ylim(-0.0012, 0.0164) ax1.set_xlabel("비트 폭 b", fontsize=14) ax1.set_ylabel("\$f(b)\$", fontsize=15) ax1.tick_params(axis="both", labelsz=12) ax1.grid(True, alpha=0.3) # ----- # 비트 폭 y축: 도함수 # ----- ax2 = ax1.twinx() line_df, = ax2.plot(b, df, linewidth=3, label="\$f'(b)=-2Aln(2)^{-1}2b\$?",) # 도함수 y=0 사이 음영 shade = ax2.fill_between(b, 0, df, alpha=0.14, label="\$\pm\$도함수 아래 음영",) ax2.set_ylim(-0.0225, 0.0012) ax2.set_ylabel("\$f'(b)\$", fontsize=15) ax2.tick_params(axis="y", labelsz=12) </pre>

```
# =====
# 4. 주석 추가
# =====

# 함수의 주석
ax1.annotate(
    r"$f(x) = 2^{(1-12x)} \text{approx } (0.2e)^x$,",
    xy=(0, 0),
    xytext=(5, 1, 0.0019),
    fontsize=13,
    arrowprops={
        "arrowstyle": ">",
        "rwidth": 1.5,
    },
),

# 도함수의 주석
ax2.annotate(
    r"$f'(x) = 2^{(1-12x)} \text{approx } (0.2e)^x$,",
    xy=(0, 0),
    xytext=(4, 0.5, 0.0031),
    fontsize=13,
    arrowprops={
        "arrowstyle": ">",
        "rwidth": 1.5,
    },
),

# =====
# 5. 도표의 완성
# =====

ax1.set_title(
    "양자화 오차 함수와 도함수의 기본 그래프",
    fontsize=16,
    pad=12,
)

# 두 축의 범례를 하나로 합침
handles = [
    line_f,
    line_tangent,
    line_of,
    vertical_line,
    point,
    shade,
]

labels = [
    r"$f(x) = A^{(1-2bx)}$",
    r"$f'(x) = bA^{(1-2bx)} \ln(A) = bA^{(1-2bx)}$",
    r"$f'(x) = 2^{(1-12x)} \ln(2) \approx 0.693 \cdot 2^{(1-12x)}$",
    r"$f(x) = 2^{(1-12x)}$",
    "점",
    "도함수 그래프 영역",
]

ax1.legend(
    handles,
    labels,
    loc="upper right",
    fontsize=11,
    frameon=True,
)

# =====
# 6. 저장 및 출력
# =====

plt.tight_layout()

output_path = Path(
    "quantization_error_function_derivative_reference.png"
)

plt.savefig(
    output_path,
    dpi=300,
    bbox_inches="tight",
)

plt.show()

print(f"그래프 저장 경로: {output_path.resolve()}")
print(f"$f(x) = (0.2e)^x$")
print(f"$f'(x) = (0.2e)^x$")
```

1. Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., & Kalenichenko, D. (2018). 「정수 연산만을 이용한 효율적인 추론을 위한 신경망 양자화와 학습」(*Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference*). *IEEE/CVF 컴퓨터 비전 및 패턴 인식 학술대회 (CVPR)*, 2704–2713.

균일 양자화, scale 설정, 저비트 정수 표현의 기본 원리를 참고함.

2. Li, X. L., Thickstun, J., Gulrajani, I., Liang, P., & Hashimoto, T. B. (2022). 「제어 가능한 텍스트 생성을 향상시키는 디퓨전 언어모델」(*Diffusion-LM Improves Controllable Text Generation*). *Advances in Neural Information Processing Systems*, 35.

연속적인 표현 공간에서 확산 과정을 이용하여 문장을 생성하는 디퓨전 언어모델의 기본 구조를 참고함.

3. Li, X. L., Thickstun, J., Gulrajani, I., Liang, P., & Hashimoto, T. B. (2022). 「제어 가능한 텍스트 생성을 향상시키는 디퓨전 언어모델」(*Diffusion-LM Improves Controllable Text Generation*). *Advances in Neural Information Processing Systems*, 35.

연속적인 표현 공간에서 확산 과정을 이용하여 문장을 생성하는 디퓨전 언어모델의 기본 구조를 참고함.

4. Python Software Foundation. Python 프로그래밍 언어
5. Julia Software Foundation. Julia 프로그래밍 언어